UNIVERSIDADE FEDERAL DO CEARÁ

Relatório Final - Refatoração de Test Smells

Disciplina: Qualidade de Software (2026.2 - T01)
Equipe: Felipe da Silva Maia (514914) e Guilherme Sousa (565684)
Projeto: PipesHub (https://github.com/pipeshub-ai/pipeshub-ai)
Fork: https://github.com/fel0psmya/pipeshub-ai
Pull Request: refactor(tests): remove test smells from unit modules by fel0psmya · Pull Request #4946 · pipeshub-ai/pipeshub-ai
Commit SHA: 93e6bd99a854071f57a8dc5b47ddc3b13f7e2e2d
Diário de Bordo: Link para o Diário de Bordo.

1. Apresentação do projeto

O `PipesHub` é uma plataforma open-source de contexto e inteligência artificial para ambientes corporativos. Seu objetivo principal é conectar sistemas e bases de conhecimento da empresa (como `Slack`, `Google Drive`, `GitHub`, `Jira`, `Microsoft 365` e `Notion`) a aplicações de IA e agentes, fornecendo uma camada de busca unificada, governada e orientada a permissões de acesso com citação de fontes.
A estrutura de testes do projeto está organizada principalmente em torno da suíte de testes em `Python` localizada no diretório de testes do repositório, utilizando o ecossistema `Pytest` para execução e verificação, além do suporte a testes de integração para os conectores, SDKs e serviços backend da aplicação.

2. Estado inicial do projeto

2.1 Test smells detectados antes da refatoração (TEMPY)

Test smell	Ocorrências detectadas
Unknown Test	1.343
Programming Paradigms Blend	1.197
Conditional Test Logic	1.159
Exception Handling	161
Undefined Test	115
Verbose Test	114
Non-Functional Statement	99
Sleepy Test	13

Test smell	Ocorrências detectadas
Total	**4.201**

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

(Nota metodológica: na execução local em Windows com `Python 3.12` sobre o escopo `tests/unit/modules`, além dos valores abaixo, o `Pytest` contabilizou 7 `xfailed`, totalizando 7.563 testes coletados).

Métrica	Valor
Testes aprovados (Passed)	7443
Testes com falha (Failed)	43 (falhas de ambiente Windows preexistentes)
Erros (Errors)	0 (com exclusão de test_parse_pool.py, exclusivo para Unix)
Ignorados (Skipped)	70
Cobertura total (%)	97.2%

2.3 Métricas do código de teste antes da refatoração (Radon)

Nota de escopo: o script padrão fornecido na disciplina executa o Radon sobre `./app`, gerando nos CSVs da pasta `metrics-before-radon/` os valores globais de SLOC = 761.145, CC média = 5,60 e MI médio = 65,50. Como o código de produção não foi alterado, esses valores permaneceram idênticos no pós-refatoração. Para atender à instrução do relatório de avaliar especificamente o código de teste, a tabela abaixo e a Seção 4 apresentam as métricas extraídas pelo Radon **diretamente sobre o escopo de testes trabalhado em `tests/unit/modules/`**

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	69.136 (nos 13 arquivos alterados: 12.684)
Complexidade ciclomática média das funções de teste	2,6808 (padrão Radon com `assert`; 1,3176 sem contar `assert`)
Índice de manutenibilidade médio dos arquivos de teste	40,2894 (nos 13 arquivos alterados: 24,396)

3. Refatorações realizadas

3.1 Seleção das ocorrências

Foram selecionadas 49 ocorrências alvo distribuídas em 6 tipos diferentes de `test smells` ao longo de 13 arquivos de teste no escopo `backend/python/tests/unit/modules/`:

- **Unknown Test** (19 ocorrências alvo + 41 eliminadas em cascata = 60 no total): Priorizado por afetar diretamente a Confiabilidade e a Manutenibilidade segundo o `CATSPY`. Testes sem asserções explícitas (que usam apenas `pytest.raises` sem inspecionar a exceção capturada, ou métodos vazios) podem passar mesmo quando o código sob teste falha por motivos inesperados.
- **Conditional Test Logic** (12 ocorrências): Possui alta severidade por impactar simultaneamente Legibilidade, Manutenibilidade e Confiabilidade. Laços `for` e compreensões de lista com filtros (`if`) dentro de testes ocultam qual caso específico falhou, interrompem a validação dos casos seguintes no primeiro erro e podem mascarar erros adicionais.
- **Non-Functional Statement** (8 ocorrências): Impacta Manutenibilidade e Confiabilidade. Instruções `pass` em métodos de teste geram falsos positivos (testes que passam sem validar nada), enquanto em classes auxiliares de protocolo ou comentários exigem limpeza ou conversão da expectativa descrita em asserção explícita.
- **Exception Handling** (5 ocorrências): Impacta Legibilidade e Confiabilidade. O uso manual de blocos `try/except` ou `try/finally` dentro de testes ofusca a intenção do teste e substitui mecanismos nativos e mais seguros do framework (`pytest.raises`, fixtures com `yield` e `tmp_path`).
- **Undefined Test** (3 ocorrências): Impacta Manutenibilidade e Confiabilidade. Funções auxiliares contendo asserções fora das convenções de nomenclatura confundem ferramentas de análise estática (como o `TEMPY`) e desenvolvedores sobre o que é um teste executável e o que é um helper.
- **Verbose Test** (2 ocorrências): Impacta Legibilidade e Manutenibilidade. Métodos longos (46 a 47 linhas) sobrecarregados com preparação repetitiva de mocks escondem o comportamento real verificado pelo teste.

3.2 Lista das refatorações

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	Non-Functional Statement	`parsers/test_excel_parser.py::test_mm_ss_with_time_context`	`c3e73ec`	sim	sim	não
R02	Unknown Test	`parsers/test_excel_parser.py::test_mm_ss_with_time_context`	`c3e73ec`	sim	sim	não
R03	Unknown Test	`parsers/test_markdown_parser.py::test_parse_file_failure_raises_value_error`	`096ab69`	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R04	Unknown Test	`parsers/test_markdown_parser.py::test_parse_file_partial_success_raises`	`64599e3`	sim	sim	não
R05	Unknown Test	`parsers/test_markdown_parser.py::test_parse_file_error_status`	`3872f95`	sim	sim	não
R06	Unknown Test	`parsers/test_markdown_parser.py::test_parse_file_empty_status`	`431c8cb`	sim	sim	não
R07	Non-Functional Statement	`parsers/test_excel_parser.py::test_formula_cell_data_type_f`	`1504910`	sim	sim	não
R08	Unknown Test	`parsers/test_excel_parser.py::test_formula_cell_data_type_f`	`1504910`	sim	sim	não
R09	Unknown Test	`parsers/test_excel_parser.py::test_process_sheet_error_propagates`	`ce46013`	sim	sim	não
R10	Unknown Test	`parsers/test_excel_parser.py::test_process_cell_error_propagates`	`2d06e84`	sim	sim	não
R11	Non-Functional Statement	`parsers/test_markdown_parser.py::test_protocol_extract_and_replace_images_body`	`4556c32`	sim	sim	não
R12	Unknown Test	`parsers/test_remaining_parsers.py::TestDocParser.test_libreoffice_not_installed_raises`	`fe1a4fe`	sim	sim	não
R13	Unknown Test	`parsers/test_remaining_parsers.py::TestDocParser.test_conversion_file_not_found_raises`	`fa59a17`	sim	sim	não
R14	Non-Functional Statement	`agents/context/test_tool_descriptions.py::test_annotation_without_origin_or_name`	`2afd04c`	sim	sim	não
R15	Exception Handling	`indexing/test_rate_limit_deferral.py::test_found_under_a_wrapper`	`222a1b8`	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R16	Exception Handling	`indexing/test_rate_limit_deferral.py::test_found_under_a_wrapper`	`0201b31`	sim	sim	não
R17	Conditional Test Logic	`parsers/test_markdown_to_blocks.py::_child_block_indices`	`95a0bab`	sim	sim	não
R18	Undefined Test	`parsers/test_markdown_to_blocks.py::_child_block_indices`	`732c3cd`	sim	sim	não
R19	Conditional Test Logic	`parsers/test_markdown_to_blocks.py::_child_group_indices`	`1f5b7e4`	sim	sim	não
R20	Undefined Test	`parsers/test_markdown_to_blocks.py::_child_group_indices`	`a6f5a8c`	sim	sim	não
R21	Conditional Test Logic	`parsers/test_markdown_to_blocks.py::_assert_all_blocks_have_data`	`f593777`	sim	sim	não
R22	Exception Handling	`parsers/test_opencv_layout_analyzer_cov95.py::test_render_all_pages`	`cb5c9d7`	sim	sim	não
R23	Exception Handling	`parsers/test_opencv_layout_analyzer_cov95.py::test_rasterize_page_with_pdf_path`	`68d1185`	sim	sim	não
R24	Conditional Test Logic	`parsers/test_excel_parser.py::test_values_are_strings`	`cf416b7`	sim	sim	não
R25	Conditional Test Logic	`parsers/test_excel_parser.py::test_all_entries_are_tuples`	`3c047e4`	sim	sim	não
R26	Non-Functional Statement	`parsers/test_html_parser_shim.py::test_protocol_clean_html_body`	`88ed089`	sim	sim	não
R27	Non-Functional Statement	`parsers/test_html_parser_shim.py::test_protocol_replace_relative_image_urls_body`	`a43fcc9`	sim	sim	não
R28	Non-Functional Statement	`parsers/test_html_parser_shim.py::test_protocol_extract_and_replace_images_body`	`c43e006`	sim	sim	não
R29	Exception Handling	`parsers/test_pdf_rasterizer.py::test_broken_process_pool`	`d7d9f40`	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
		_reports_memory_incident_when_governor_wired				
R30	Conditional Test Logic	parsers/test_pdf_rasterizer.py::test_effective_resolution_leaves_standard_pages_untouched	814eaaa	sim	sim	não
R31	Conditional Test Logic	agents/qna/test_schemas.py::test_all_confidence_values	32dcf6d	sim	sim	não
R32	Conditional Test Logic	agents/qna/test_schemas.py::test_all_answer_match_types	7037c31	sim	sim	não
R33	Conditional Test Logic	agents/qna/test_schemas.py::test_all_answer_match_types_are_defined	c574aa6	sim	sim	não
R34	Unknown Test	agents/qna/test_schemas.py::test_invalid_confidence_raises	35aba66	sim	sim	não
R35	Unknown Test	agents/qna/test_schemas.py::test_invalid_answer_match_type_raises	cc23017	sim	sim	não
R36	Unknown Test	agents/qna/test_schemas.py::test_missing_required_answer_raises	099e606	sim	sim	não
R37	Unknown Test	agents/qna/test_schemas.py::test_missing_required_confidence_raises	b0009ef	sim	sim	não
R38	Unknown Test	parsers/test_pptx_parser.py::test_different_status_values_fail	50b3cc5	sim	sim	não
R39	Unknown Test	parsers/test_pptx_parser.py::TestParseBinary.test_failure_raises_value_error	63d91fd	sim	sim	não
R40	Unknown Test	parsers/test_pptx_parser.py::test_converter_exception_propagates	98a80d5	sim	sim	não
R41	Unknown Test	parsers/test_pptx_parser.py::TestParseFile.test_failure_raises_value_error	b9f3ba2	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R42	Unknown Test	`parsers/test_pptx_parser.py::test_file_not_found_propagates`	`b563d0b`	sim	sim	não
R43	Non-Functional Statement	`parsers/test_pdf_processors_extended.py::test_detect_text_regions_filters_small_and_in_table`	`04ebb62`	sim (skip)	sim	não
R44	Verbose Test	`parsers/test_pdf_processors_extended.py::test_analyze_page_text_region_classified_as_text`	`31ef8b0`	sim (skip)	sim	não
R45	Verbose Test	`parsers/test_pdf_processors_extended.py::test_analyze_page_unclaimed_block_as_heading`	`5eabc5f`	sim (skip)	sim	não
R46	Undefined Test	`transformers/test_block_container_validator.py::_assert_raises_with_codes`	`eacb10e`	sim	sim (+41 cascata)	não
R47	Conditional Test Logic	`transformers/test_block_container_validator.py::test_children_block_range_huge_end_single_error`	`ef2640b`	sim	sim	não
R48	Conditional Test Logic	`transformers/test_block_container_validator.py::test_children_group_range_huge_end_single_error`	`db093d8`	sim	sim	não
R49	Conditional Test Logic	`parsers/test_pptx_parser.py::test_different_status_values_fail`	`d870680`	sim	sim	não

3.3 Exemplos detalhados

R24 – Conditional Test Logic (`parsers/test_excel_parser.py::test_values_are_strings`)

Antes:

```
388     def test_values_are_strings(self):
389         for code, fmt in BUILTIN_DATE_FORMATS.items():
390             assert isinstance(fmt, str), f"Code {code} has non-string format"
```

Depois:

```
388     @pytest.mark.parametrize("code, fmt", BUILTIN_DATE_FORMATS.items())
389     def test_values_are_strings(self, code, fmt):
390         assert isinstance(fmt, str), f"Code {code} has non-string format"
```

Por que era um test smell: O método utilizava um laço `for` para percorrer todas as 9 entradas de `BUILTIN_DATE_FORMATS` e verificar com `assert` se cada valor era uma string. Caso uma entrada falhasse, as demais não seriam avaliadas e o relatório não identificaria diretamente qual código falhou.

Estratégia do CATSPY aplicada: O `CATSPY` recomenda extrair cada caso condicional em um método de teste próprio. Para evitar duplicar o método 9 vezes manualmente, adaptou-se a estratégia para `@pytest.mark.parametrize("code,fmt", BUILTIN_DATE_FORMATS.items())`, gerando um caso de teste independente por entrada.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	3 (2 s/ `assert`)	2 (1 s/ `assert`)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	2315	2301

R34 – Unknown Test (`agents/qna/test_schemas.py::test_invalid_confidence_raises`)

Antes:

```
117     def test_invalid_confidence_raises(self):
118         with pytest.raises(ValidationError):
119             AgentAnswerWithMetadataJSON(
120                 answer="x",
121                 confidence="Ultra",  # not a valid Literal
122                 answerMatchType="Exact Match",
123             )
```

Depois:

```python
    def test_invalid_confidence_raises(self):
        with pytest.raises(ValidationError) as exc_info:
            AgentAnswerWithMetadataJSON(
                answer="x",
                confidence="Ultra",  # not a valid Literal
                answerMatchType="Exact Match",
            )

        errors = exc_info.value.errors()
        assert len(errors) == 1
        assert errors[0]["loc"] == ("confidence",)
        assert errors[0]["type"] == "literal_error"
```

Por que era um test smell: O método instanciava `AgentAnswerWithMetadataJSON` com `confidence="Ultra"` dentro de `pytest.raises(ValidationError)`, mas sem nenhuma instrução `assert` explícita. Confirmava apenas que algum erro de validação ocorria, sem validar qual campo ou tipo de erro o causou.

Estratégia do CATSPY aplicada: Introdução de asserções explícitas conforme o `CATSPY`, adaptada ao `pytest`: captura da exceção com `as exc_info` e validação explícita de que há exatamente 1 erro, com `loc == ("confidence",)` e `type == "literal_error"`.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	4 (1 s/ `assert`)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	164	184

R15 e R16 – Exception Handling (`indexing/test_rate_limit_deferral.py::test_found_under_a_wrapper`)

Antes:

```python
    def test_found_under_a_wrapper(self) -> None:
        try:
            try:
                raise _limited(600)
            except ApiCallError as inner:
                raise RuntimeError("download failed") from inner
        except RuntimeError as outer:
            assert rate_limit_retry_after(outer) == 600
```

Depois:

```
38      def test_found_under_a_wrapper(self) -> None:
39          inner = _limited(600)
40
41          with pytest.raises(RuntimeError) as exc_info:
42              raise RuntimeError("download failed") from inner
43
44          assert rate_limit_retry_after(exc_info.value) == 600
```

Por que era um test smell: O teste utilizava dois blocos `try/except` aninhados: o interno para construir manualmente uma cadeia de exceções (`raise ... from`) e o externo para capturar o `RuntimeError` e verificá-lo com `assert`, ofuscando a intenção do teste.

Estratégia do CATSPY aplicada: Substituição do `try/except` externo por `with pytest.raises(RuntimeError) as exc_info` (equivalente em `pytest` ao `assertRaises` indicado pelo `CATSPY`) e remoção do `try/except` interno lançando diretamente `raise RuntimeError("download failed") from _limited(600)`.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	4 (3 s/ assert)	2 (1 s/ assert)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	100	97

R01 – Non-Functional Statement (`parsers/test_excel_parser.py::test_mm_ss_with_time_context`)

Antes:

```
2635    def test_mm_ss_with_time_context(self):
2636        """mm:ss format where has_time detects time (via h in format), covering line 251.
2637
2638        When the format has 'h' making has_time=True and then mm:ss appears with no
2639        h patterns in the python_format, the special case branch is taken.
2640        """
2641        # This is a tricky case. We need has_time=True but no 'h' in python_format.
2642        # Actually, looking at line 248-251 more carefully:
2643        # has_time checks for 'h' in format_str (the original)
2644        # Line 250 checks for [hH%] in python_format (after substitutions)
2645        # If format_str has 'h' but the h gets consumed, it might not be present
2646        # However, this is hard to trigger naturally. Instead:
2647        # "mm:ss" has no 'h' so has_time=False, taking the else branch (line 263-265).
2648        # For coverage of line 251, we need a format where has_time=True but
2649        # python_format after step 5 has NO [hH%] -> this is the mm:ss without hours path.
2650        pass
```

Depois:

Código removido.

Por que era um test smell: O método continha apenas docstring, comentários e a instrução `pass`, não exercitando o código sob teste e gerando falso positivo nos resultados da suíte.

Estratégia do CATSPY aplicada: Remoção do método de teste sem finalidade (uma das duas opções indicadas pelo `CATSPY`), pois a análise de `_resolve_ambiguous_format` demonstrou que o ramo que o teste pretendia cobrir é inalcançável.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	0 (método removido)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	2315	2301

R46 – Undefined Test (`transformers/test_block_container_validator.py::_assert_raises_with_codes`)

Antes:

```
159  def _assert_raises_with_codes(container: BlocksContainer, *codes: str) -> None:
160      with pytest.raises(BlockContainerValidationError) as exc_info:
161          _validator().validate(container)
162      assert _error_codes(exc_info.value) == set(codes)
```

Depois:

```
159  def assert_raises_with_codes(container: BlocksContainer, *codes: str) -> None:
160      with pytest.raises(BlockContainerValidationError) as exc_info:
161          _validator().validate(container)
162      assert _error_codes(exc_info.value) == set(codes)
```

Por que era um test smell: A função auxiliar continha asserções (`pytest.raises` e `assert`), mas seu nome começava com `_assert`, fazendo com que o `TEMPY` a reportasse como `Undefined Test` e não a reconhecesse como asserção customizada nos 41 testes que a chamavam.

Estratégia do CATSPY aplicada: Renomeação do método para a convenção de asserção customizada (`assert_raises_with_codes`) e atualização das 42 chamadas no arquivo, eliminando o `Undefined Test` e, em cascata, 41 ocorrências de `Unknown Test`.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2 (1 s/ `assert`)	2 (1 s/ `assert`)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	891	888

R44 – Verbose Test (`parsers/test_pdf_processors_extended.py::test_analyze_page_text_region_classified_as_text`)

Antes:

```python
    @patch("app.modules.parsers.pdf.opencv_layout_analyzer.cv2")
    def test_analyze_page_text_region_classified_as_text(self, mock_cv2):
        """analyze_page classifies text region as TEXT when not heading or list."""
        analyzer = self._make_analyzer()

        # Mock page
        mock_page = MagicMock()
        mock_page.rect.width = 612.0
        mock_page.rect.height = 792.0

        # Render returns an image
        mock_pix = MagicMock()
        mock_pix.height = 100
        mock_pix.width = 80
        mock_pix.samples = np.zeros(100 * 80 * 3, dtype=np.uint8).tobytes()
        mock_page.get_pixmap.return_value = mock_pix

        # OpenCV mocks
        mock_cv2.cvtColor.return_value = np.zeros((100, 80), dtype=np.uint8)
        mock_cv2.adaptiveThreshold.return_value = np.zeros((100, 80), dtype=np.uint8)
        mock_cv2.morphologyEx.return_value = np.zeros((100, 80), dtype=np.uint8)
        mock_cv2.getStructuringElement.return_value = np.ones((1, 10), dtype=np.uint8)
        mock_cv2.add.return_value = np.zeros((100, 80), dtype=np.uint8)
        mock_cv2.dilate.return_value = np.zeros((100, 80), dtype=np.uint8)

        # No table contours, no text contours from CV2
        mock_cv2.findContours.return_value = ([], None)

        # get_images returns empty
        mock_page.get_images.return_value = []

        # PyMuPDF text dict: one text block
        mock_page.get_text.return_value = {
            "blocks": [{
                "type": 0,
                "bbox": (50, 50, 200, 100),
                "lines": [{
                    "spans": [{"text": "Normal paragraph text here.", "size": 12.0, "flags": 0}]
                }]
            }]
        }

        regions = analyzer.analyze_page(mock_page)
        # The block should be picked up by _collect_unclaimed_text_blocks as TEXT
        text_regions = [r for r in regions if r.type.value == "text"]
        assert len(text_regions) >= 1
        assert "Normal paragraph text here." in text_regions[0].text

```

Depois:

```python
    @staticmethod
    def _make_blank_page():
        """Letter-size page that renders to a blank 80x100 image and has no images."""
        mock_page = MagicMock()
        mock_page.rect.width = 612.0
        mock_page.rect.height = 792.0
        mock_pix = MagicMock()
        mock_pix.height = 100
        mock_pix.width = 80
        mock_pix.samples = np.zeros(100 * 80 * 3, dtype=np.uint8).tobytes()
        mock_page.get_pixmap.return_value = mock_pix
        mock_page.get_images.return_value = []
        return mock_page

    @staticmethod
    def _stub_cv2_without_contours(mock_cv2):
        """OpenCV finds no table or text contours, so every text block stays unclaimed."""
        mock_cv2.cvtColor.return_value = np.zeros((100, 80), dtype=np.uint8)
        mock_cv2.adaptiveThreshold.return_value = np.zeros((100, 80), dtype=np.uint8)
        mock_cv2.morphologyEx.return_value = np.zeros((100, 80), dtype=np.uint8)
        mock_cv2.getStructuringElement.return_value = np.ones((1, 10), dtype=np.uint8)
        mock_cv2.add.return_value = np.zeros((100, 80), dtype=np.uint8)
        mock_cv2.dilate.return_value = np.zeros((100, 80), dtype=np.uint8)
        mock_cv2.findContours.return_value = ([], None)
```

```python
    @patch("app.modules.parsers.pdf.opencv_layout_analyzer.cv2")
    def test_analyze_page_text_region_classified_as_text(self, mock_cv2):
        """analyze_page classifies text region as TEXT when not heading or list."""
        analyzer = self._make_analyzer()
        self._stub_cv2_without_contours(mock_cv2)
        mock_page = self._make_blank_page()
        mock_page.get_text.return_value = {
            "blocks": [{
                "type": 0,
                "bbox": (50, 50, 200, 100),
                "lines": [{
                    "spans": [{"text": "Normal paragraph text here.", "size": 12.0, "flags": 0}]
                }]
            }]
        }

        regions = analyzer.analyze_page(mock_page)
        # The block should be picked up by _collect_unclaimed_text_blocks as TEXT
        text_regions = [r for r in regions if r.type.value == "text"]
        assert len(text_regions) >= 1
        assert "Normal paragraph text here." in text_regions[0].text
```

Por que era um test smell: O método possuía 46 linhas, sendo a maior parte dedicada à montagem repetitiva de uma página PDF falsa e à configuração do `OpenCV` sem contornos, escondendo o cenário principal verificado pelo teste.

Estratégia do CATSPY aplicada: Extração da preparação comum para métodos auxiliares reutilizáveis (`_make_blank_page` e `_stub_cv2_without_contours`), reduzindo o método de 46 para 20 linhas e reutilizando os mesmos helpers no R45.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	5 (3 s/ `assert`)	5 (3 s/ `assert`)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	758	752

4. Estado final e comparação

Escopos medidos de forma pareada Antes/Depois: TEMPY sobre a suíte completa em `backend/python/tests/` — sendo que em `tests/unit/modules/` os smells caíram de 596 para 506, a mesma variação de -90; Pytest e Radon sobre a suíte de testes unitários em `backend/python/tests/unit/modules/`

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	4.201	4.111	-90 (-2,14%)
Testes aprovados	7.443	7.500	+57
Testes com falha	43	42	-1
Cobertura total (%)	97.2%	97.2%	0
SLOC dos testes	69.136	69.131	-5
Complexidade ciclomática média das funções de teste	2,6808 (1,3176 s/ `assert`)	2,6837 (1,3159 s/ `assert`)	+0,0029 (-0,0017 s/ `assert`)
Índice de manutenibilidade médio dos arquivos de teste	40,2894	40,2343	−0,0551

Test smell	Antes	Depois	Variação
Unknown Test	1.343	1.283	-60
Programming Paradigms Blend	1.197	1.197	0
Conditional Test Logic	1.159	1.147	-12
Exception Handling	161	156	-5
Undefined Test	115	112	-3
Verbose Test	114	112	-2
Non-Functional Statement	99	91	-8
Sleepy Test	13	13	0
Total	4.201	4.111	-90

5. Análise dos resultados

Quais métricas melhoraram e quais pioraram?

A quantidade total de `test smells` no projeto caiu em 90 ocorrências (de 4.201 para 4.111), sendo que nos 13 arquivos trabalhados pela equipe todas as 49 ocorrências alvo e mais 41 ocorrências em cascata foram eliminadas. O volume de código dos 13 arquivos de teste foi reduzido em 5 linhas de SLOC no Radon (e 16 linhas totais no `git diff`, considerando 260 inserções e 276 deleções), a complexidade ciclomática das funções que possuíam laços `for`, condicionais `if` e blocos `try/except` caiu para 1, e o número de testes coletados e executados subiu em +56 casos líquidos (de 7.563 para 7.619) graças à parametrização. O código de produção (`app/`) permaneceu 100% intocado, e a leve oscilação na CC bruta (+0,0029) e no MI (-0,0551) deve-se apenas ao fato de o Radon contabilizar cada novo assert adicionado nos Unknown Tests como +1 ponto de decisão (tanto que, desconsiderando assert, a complexidade lógica média **caiu** de 1,3176 para 1,3159)

Alguma refatoração introduziu novos test smells? Quais?

Não. Cada arquivo modificado foi verificado individualmente com o `TEMPY` após cada commit. Quando uma tentativa inicial de usar list comprehension nas asserções do R34 acionou `Conditional Test Logic` no `TEMPY`, ela foi imediatamente substituída por asserções diretas antes do commit, garantindo zero smells novos introduzidos.

O resultado dos testes ou a cobertura mudou?

O comportamento funcional validado pelos testes foi integralmente preservado (tornando-se mais rigoroso nas asserções de exceções e listas de erros). A contagem de testes coletados aumentou exatamente na proporção esperada (+56 casos líquidos: -2 testes vazios removidos em `test_excel_parser.py` e +58 casos gerados por `@pytest.mark.parametrize` em 7 métodos). A variação de 43 para 42 falhas deveu-se a 1 teste instável de ambiente fora dos arquivos modificados.

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

Os mais fáceis foram `Non-Functional Statement` em subclasses locais de `Protocol` (substituição de `pass` por docstring) e `Conditional Test Logic` em laços `for` sobre constantes (conversão direta para `@pytest.mark.parametrize`). Os mais difíceis foram:

1. `Non-Functional Statement` e `Unknown Test` em métodos vazios (`test_mm_ss_with_time_context`), que exigiram inspecionar o código de produção (`_resolve_ambiguous_format`) para comprovar que o ramo era inalcançável;
2. `Exception Handling` com `try/finally` para limpeza de estado global (`_resource_governor`) ou encadeamento de exceções, resolvidos com fixtures com `yield` e `raise ... from`;
3. `Undefined Test` em helpers, que exigiram analisar as regras do detector AST do `TEMPY` para eliminar os falsos positivos sem quebrar a coleta do `pytest`.

6. Uso do CATSPY

Como você usou o CATSPY durante a atividade?

O `CATSPY` foi consultado antes de cada refatoração para confirmar a definição formal do `smell` apontado pelo `TEMPY`, revisar os impactos negativos nas dimensões de Legibilidade, Manutenibilidade e Confiabilidade, verificar se a ocorrência era um falso positivo e

selecionar a estratégia de refatoração adequada sem introduzir outros smells (como Redundant Assertion ou Test Redundancy).

Quais partes do catálogo mais ajudaram (definição, efeitos negativos, exemplo com problema, exemplo refatorado)?

As seções de Definição e Efeitos Negativos foram as que mais ajudaram para fundamentar tecnicamente as decisões no Diário de Bordo e no Pull Request, enquanto os exemplos refatorados guiaram o princípio geral de cada transformação (como extrair setup repetido em Verbose Test e isolar casos em Conditional Test Logic).

Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?

Sim, vários casos exigiram adaptação ao pytest:

1. Em Unknown Test com pytest.raises(..., match=...), capturamos a exceção com as exc_info, removemos o match= e adicionamos assert explícito sobre atributos estruturados da exceção fora do bloco with;
2. Em Conditional Test Logic sobre constantes de 9 ou 35 itens, usamos @pytest.mark.parametrize em vez de duplicar métodos manualmente, e usamos list comprehensions em helpers de dados;
3. Em Exception Handling com try/finally usado para limpeza de arquivos ou mocks globais, aplicamos a fixture nativa tmp_path e fixture com yield;
4. Em falsos positivos de Non-Functional Statement e Undefined Test, substituímos pass em classes de protocolo por docstring, convertemos comentário terminado em "pass" em asserção e ajustamos os helpers.

O que faltou ou poderia ser melhorado no catálogo?

O catálogo é fortemente orientado ao estilo unittest/xUnit e poderia incluir exemplos idiomáticos de pytest, tais como o uso de @pytest.mark.parametrize para Conditional Test Logic, pytest.raises(...) as exc_info e fixtures (tmp_path e yield) para Exception Handling, além de critérios que diferenciem métodos de teste reais de funções auxiliares (helpers) e classes de stub locais.

7. Divisão do trabalho

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Felipe da Silva Maia (514914)	R01 a R25 (em 7 arquivos e 23 commits)	25
Guilherme Sousa (565684)	R26 a R49 (em 6 arquivos e 24 commits)	24

8. Dificuldades e lições aprendidas

Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

- **Restrições de sistema operacional no Windows:** `test_parse_pool.py` importa o módulo `resource` (exclusivo de Unix) e 42-43 testes dependem de binários Linux/externos (LibreOffice e CairoSVG), o que exigiu configurar `PYTEST_ADDOPTS="--ignore=tests/unit/modules/parsers/test_parse_pool.py --timeout=300 --continue-on-collection-errors"` no ambiente sem alterar o script de extração.
- **Erro de codificação em script de medição:** Erro `KeyError: 'mi'` no script do `Radon` causado por um caractere invisível BOM (`U+FEFF`) na primeira linha de `app/agents/actions/jira/jira.py`, exigindo remover o BOM localmente apenas durante a medição e restaurar o arquivo depois para não tocar em `app/` no PR.
- **Limitações do detector AST do TEMPY:** O detector reportou falsos positivos em comentário terminado com a palavra "pass", em helpers de dados e em chamadas a `_assert_raises_with_codes`.

O que você aprendeu com a atividade?

Aprendemos na prática como `test smells` comprometem a confiabilidade e o diagnóstico de falhas de uma suíte real, o ganho de transparência ao transformar laços em testes parametrizados com `@pytest.mark.parametrize`, a importância de validar estruturalmente as exceções capturadas e boas práticas de contribuição open-source (commits atômicos, comunicação técnica no PR e isolamento de artefatos de métricas em branch separada).

9. Contribuição (Pull Request)

Foi enviado o Pull Request #4946 ("refactor(tests): remove test smells from unit modules") ao repositório oficial `pipeshub-ai/pipeshub-ai`, reunindo 47 commits em 13 arquivos de teste (260 inserções e 276 deleções) sem qualquer alteração no código de produção (`app/`). A descrição seguiu o template oficial do projeto em inglês, apresentando a tabela consolidada das 49 ocorrências alvo (e 90 eliminadas no total), notas de comportamento (Behaviour notes) explicando a remoção dos 2 testes vazios, o saldo líquido de +56 casos coletados via parametrização e o efeito cascata de `assert_raises_with_codes`, além das evidências de execução do `Pytest` e do `TEMPY`. As pastas de métricas (`metrics-before-*` e `metrics-after-*`) foram versionadas separadamente no nosso fork dentro da branch “`metricas-finais`” e dentro do diretório “backend/python/” – onde de fato os testes em python do repositório eram feitos – para manter o PR limpo. O PR aguarda revisão dos mantenedores.

10. Observações gerais

- **Efeito cascata no TEMPY** (49 alvos -> 90 eliminados): A redução total de 90 `smells` (de 4.201 para 4.111) ocorreu porque a renomeação do helper `_assert_raises_with_codes` para `assert_raises_with_codes` (R46) em `test_block_container_validator.py` permitiu que o parser AST do `TEMPY` passasse a reconhecer as chamadas nos 41 métodos de teste daquele arquivo como asserções válidas, eliminando 41 ocorrências de `Unknown Test` em cascata além das 49 ocorrências tratadas diretamente.
- **Testes em classe ignorada (`@pytest.mark.skip`):** As refatorações R43, R44 e R45 em `test_pdf_processors_extended.py` encontram-se na classe `TestOpenCVLayoutAnalyzerAnalyzePage`, que já estava marcada com `@pytest.mark.skip` no baseline por referenciar uma classe removida do código de

produção. Essas alterações foram validadas pela coleta do `pytest` e pela reexecução do `TEMPY`.

- **Padronização de escopo das ferramentas (TEMPY, Pytest e Radon):** O TEMPY (análise estática via `AST`) pôde ser executado sobre todos os 1.779 arquivos de teste do repositório (4.201 -> 4.111 smells, variação de -90, concentrada integralmente nos 13 arquivos refatorados em `tests/unit/modules/`, onde os smells caíram de 596 para 506). Já a execução dinâmica do Pytest foi delimitada ao diretório `tests/unit/modules/` (7.563 casos coletados no baseline e 7.619 após a refatoração), pois os diretórios de integração exigem infraestrutura externa ativa (bancos de dados e APIs de terceiros) e módulos exclusivos de sistemas Unix. Por fim, como o script padrão do Radon fornecido no template apontava para `./app` (SLOC = 761.145, inalterado nos CSVs brutos), incluímos no relatório a medição específica do Radon sobre `tests/unit/modules/` para refletir com precisão o impacto real nas métricas do código de teste.